



Generación autónoma de microservicios

Del dolor al estado actual: arquitectura, orquestación, verificación real y propuesta SOTA de auto-mejora

El dolor: del síntoma a la causa raíz.

0 de 15 microservicios verificados en la práctica real: la brecha crítica entre generar sintaxis y entregar software gobernable.

EL SÍNTOMA VISIBLE

0 de 15 Verificadas

- El sistema generaba código con apariencia profesional y sintaxis limpia.
- Al ejecutar pruebas herméticas, **ningún servicio compilaba**.
- El bloqueo real: un defecto de montaje de volumen Docker y dependencias no aisladas.

LA ILUSIÓN DEL ASISTENTE

Fragmentación Ciega

- Copilot y Cursor asisten a nivel de archivo o función aislada.
- Nadie gobierna la coherencia entre capas, contratos ni dependencias.
- Generan código rápido, pero trasladan todo el coste a la **depuración humana**.

LA CAUSA RAÍZ

Falta de Gobernanza

- Sin compuertas de verificación hermética, la IA acelera deuda técnica.
- No falta capacidad lingüística en el LLM; falta **diseño por especificación**.
- La solución exige que la especificación mande y verifique antes de entregar.

Principio rector: Un generador de código que no verifica su propia salida en un entorno sellado no es una herramienta de ingeniería; es un generador de deuda técnica a velocidad de LLM.

Dónde está la industria: la paradoja de la IA.

La adopción masiva de asistentes de código no ha resuelto la entrega de software: ha desplazado el cuello de botella.

1 · ASISTENTES DE CÓDIGO

Productividad Aparente

- **84%** de desarrolladores usan herramientas de IA a diario.
- Aceleración visible en autocompletar funciones repetitivas.
- Mercado saturado y caro (\$5–\$30 por tarea compleja).

2 · EL CUELLO DE BOTELLA

La Paradoja del 76%

- El **76% del tiempo** se consume en depurar, verificar y corregir alucinaciones.
- La promesa: «*velocidad de entrega inmediata*».
- La realidad: «*deuda técnica acelerada y fatiga de revisión humana*».

3 · NUESTRO ENFOQUE

Spec-Driven & Agéntico

- Construir microservicios completos desde especificación formal.
- Orquestación multi-agente con **verificación hermética previa**.
- Solo 12–55% resuelto en la industria: liderazgo de TCS.

La lección de toda la investigación: el ingrediente que gana es una **verificación de fiar**, no un modelo más grande.

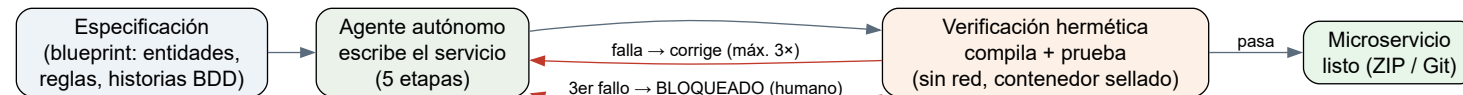
Spec-Driven Design: la especificación manda.

El modelo no inventa la arquitectura: la especificación formal es la única fuente de verdad inmutable.

Nivel de Especificación	Qué prescribe con fuerza contractual
Constitution (4 capas)	Dominio, Servicio, Infraestructura, API. Prohibición estricta de acoplamiento cruzado y llamadas circulares.
Contratos OpenAPI & BDD (19)	Definición canónica de esquemas de datos, DTOs inmutables, códigos de estado HTTP y escenarios Given-When-Then.
Lista blanca de dependencias	Solo librerías corporativas homologadas y versiones congeladas en el POM raíz. Prohibición de dependencias sorpresa.
El LLM como compilador	El modelo actúa estrictamente como un compilador de especificación a código Java ejecutable, no como arquitecto creativo.

Garantía arquitectónica: Si el modelo intenta generar un endpoint o dependencia fuera de la especificación, la compuerta rechaza el artefacto antes de compilar.

Cómo funciona la solución: arquitectura desacoplada.



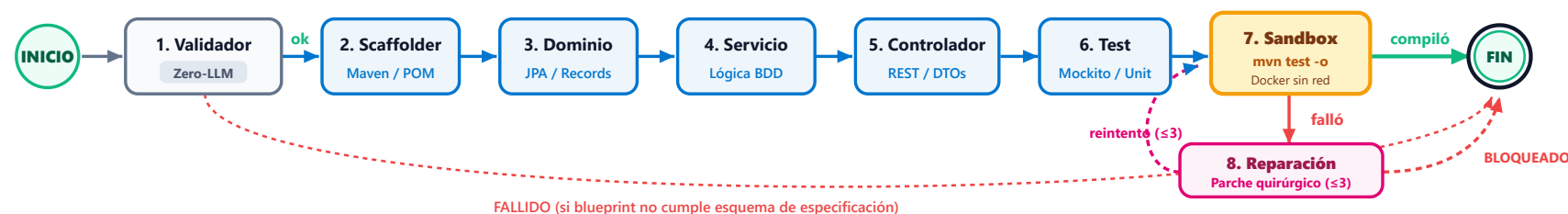
PIPELINE DESACOPLADO EN 5 CAPAS

De la Intención a la Entrega

- **1. Web Studio (React/Vite):** 10 fases interactivas, blueprint visual y streaming SSE.
- **2. API Gateway (FastAPI):** Gestión de sesiones, persistencia y control de ejecución.
- **3. Motor Agéntico (LangGraph):** Grafo coordinado de 8 agentes especializados con memoria.
- **4. Sandbox Hermético (Docker):** Contenedor sellado, red deshabilitada, caché .m2 de solo lectura.
- **5. Registro & Observabilidad:** Trazabilidad de costes y tokens en SQLite y MLflow.

Contención de fallos: Si la verificación falla, el agente de reparación aplica un parche quirúrgico (máx. 3 intentos). Si persiste, escala a estado BLOQUEADO.

Orquestación LangGraph: 8 agentes coordinados.



5 AGENTES ESPECIALIZADOS LLM

Especialistas de Dominio Técnico

- **1. Scaffolder:** Estructura Maven y dependencias de lista blanca.
- **2. Dominio:** Entidades JPA, records inmutables, repositorios.
- **3. Servicio:** Lógica transaccional y reglas de negocio.
- **4. Controlador:** Endpoints REST y @RestControllerAdvice.
- **5. Test:** Cobertura de integración y unitaria mapeada a BDD.

3 AGENTES DETERMINISTAS / GUARDAS

Gobernanza y Verificación Hermética

- **6. Validador estático:** Analiza sintaxis y esquemas JSON antes de compilar.
- **7. Sandbox Docker:** Ejecuta `mvn test -o` en contenedor sin red.
- **8. Reparación reflexiva:** Analiza traza de error y reintenta con límite estricto de **máx. 3 iteraciones** antes de escalar a humano (BLOQUEADO).

Anatomía del prompt y reglas de ingeniería.

Segregación de contexto: cada agente recibe únicamente las reglas e interfaces necesarias para su tarea.

NIVEL 1

Base Contextual

Directivas inmutables del framework:
Spring Boot 3.x, Java 21 LTS, perfil
hermético.

NIVEL 2

Reglas de Capa

Prohibición absoluta de Lombok; uso
forzoso de records para DTOs;
repositorios Spring Data JPA.

NIVEL 3

Contrato BDD

Mapeo 1:1 entre historias BDD de la
especificación y pruebas Mockito
ejecutables.

NIVEL 4

Salida Pura

Bloques de código estructurados y
esquemas JSON. Sin texto conversacional
ni explicaciones ambiguas.

Resultado directo: Cero colisiones de paquetes, código estandarizado y determinismo en compilación sin dependencias externas.

El proceso, de punta a punta.

Siete fases continuas: de los requisitos en lenguaje natural al microservicio compilado y verificado.



Lo que eso garantiza.

Compila siempre

Verificado en un contenedor sin red con Maven en modo offline. Si no compila, **no se entrega**.

Respetar la arquitectura

Separación de capas innegociable. Sin mezclar controladores con persistencia.

Pruebas que pasan

Pruebas de unidad y de integración reales, no decorativas. Creadas contra la especificación.

Mismo código, misma calidad

El flujo paso a paso y el flujo automático producen el mismo resultado fiable.

No es «código asistido por IA». Es **software corporativo construido con rigor** donde la IA es el motor, no el piloto.

La interfaz de usuario: React/Vite y 10 pestañas.

Una consola de ingeniería completa para operar, auditar y gobernar el pipeline agéntico.

10 FASES GUIADAS

Ciclo Completo

Pestañas dedicadas: Requisitos, Arquitectura, Modelos SQL, Contratos, Blueprints, hasta Generación y Sandbox.

STREAMING SSE

Monitor en Vivo

Observabilidad en tiempo real de cada agente: trazas de ejecución, grafos Mermaid dinámicos y logs de Maven.

FLEXIBILIDAD

Selector Multi-LLM

Soporte para Gemini, Groq, OpenAI, DeepSeek y Mock local determinista, configurable por sesión.

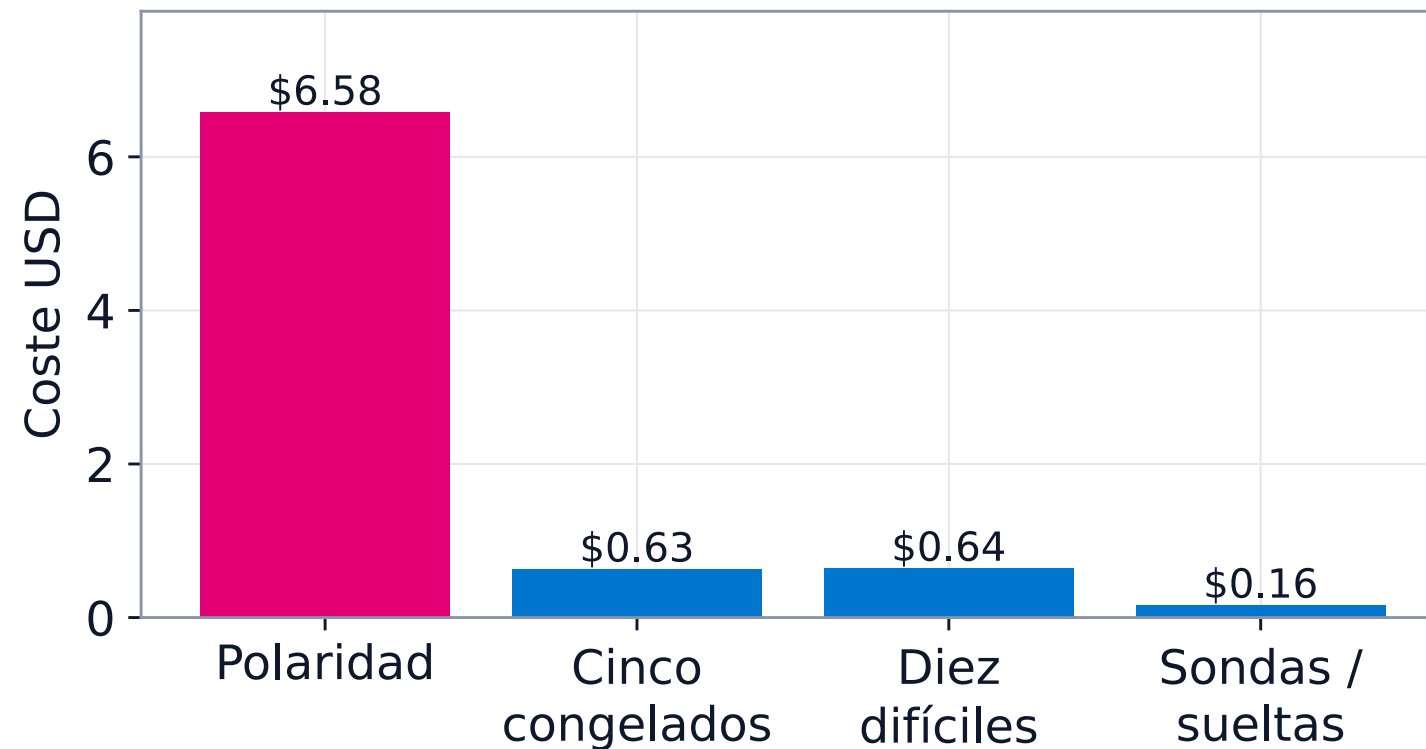
ENTREGA ATÓMICA

Descarga Directa

Exportación en ZIP listo para despliegue o publicación Git automática con verificación de 3 estados.

Transparencia absoluta: El operador humano puede inspeccionar los diagnósticos intermedios o pausar el flujo en cualquier etapa.

El coste.



\$0.06 USD

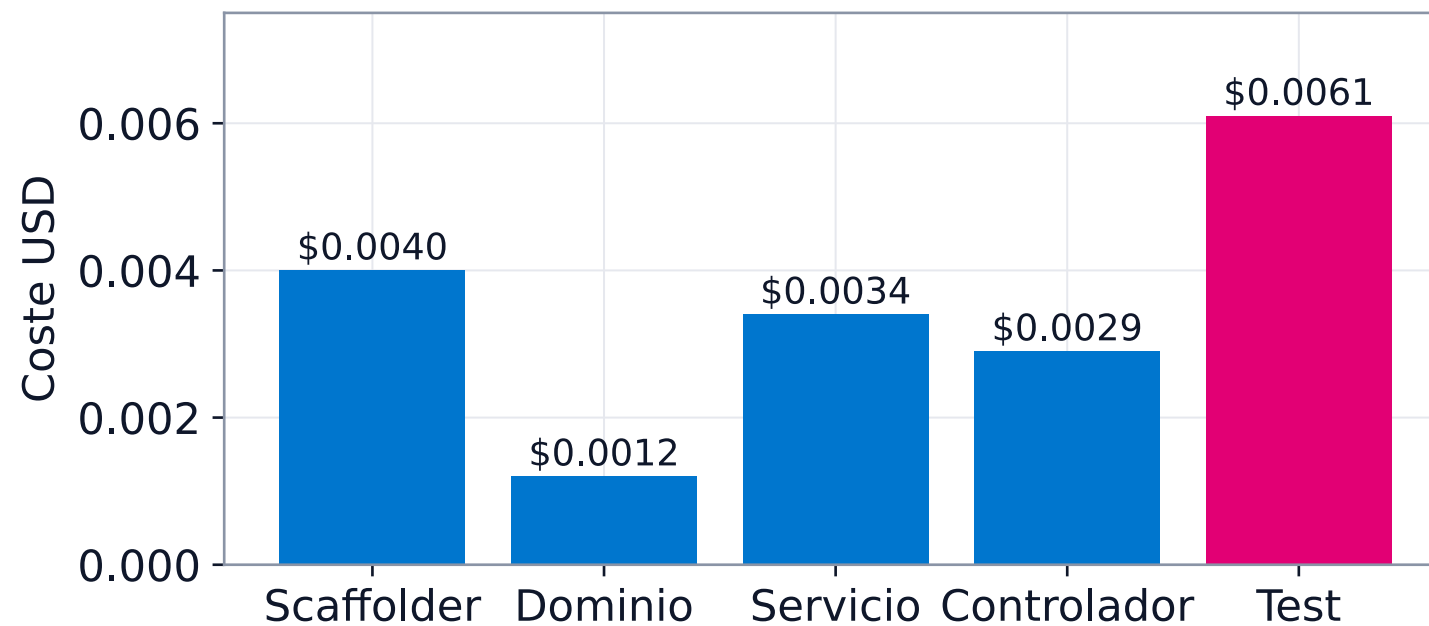
por servicio generado y verificado (modelo medio, 5 etapas).

\$0.20 USD

por servicio con **3 candidatos** compitiendo; se elige el que compila y pasa pruebas.

- 100 microservicios: **\$6 a \$20**
- 1.000 microservicios: **\$60 a \$200**

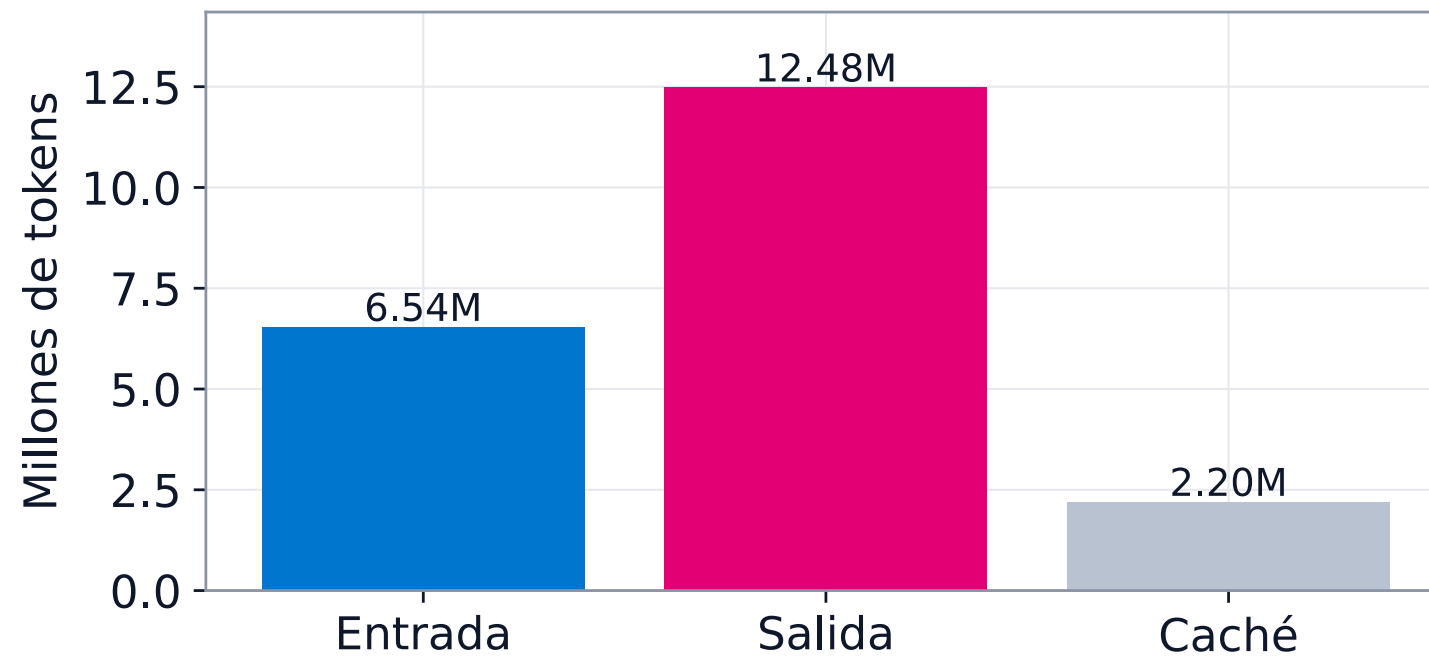
Coste por etapa.



- La **implementación** concentra el 71% del coste (es donde se escribe el código).
- Las demás etapas cuestan fracciones de centavo.
- La **verificación en contenedor no usa IA**: coste marginal cero.

Tokens

(último reporte de MLflow) ●



19.0M tokens combinados

6.5M entrada · 12.5M salida

2.2M desde caché

- **78K tokens** por servicio
- 15.6K por llamada; salida $\approx 1.9\times$ la entrada

Estado actual: de la teoría a la verificación real.

Lo que hoy está construido, probado y auditado en la plataforma Microservice Code Studio.

- **Primera compilación verificada en sandbox hermético:** corrección del defecto de montaje Docker; la plataforma ejecuta `mvn test -o sin red` y valida compilación y pruebas BDD reales.
- **Paridad absoluta entre rutas de producto:** tanto la ruta secuencial como la orquestada por LangGraph compilan, ejecutan pruebas y comparten la misma veta de verificación.
- **Métrica de calidad inmanipulable y auditable:** transición de densidad heurística a penalización absoluta contra línea base congelada; auditoría eliminó resultados fabricados.
- **Registro honesto de costes en MLflow:** seguimiento riguroso de tokens de entrada, salida y caché; identificada discrepancia del 20% para corrección en SQLite.
- **Punto abierto identificado (33% intermitencia bajo carga):** 4/6 bajo concurrencia, 8/8 secuencial; atribuido a reetiquetado SELinux :Z en volumen compartido (en resolución).

Propuesta de auto-mejora: el enfoque SkillOpt.

Aprender de los diagnósticos de fallo para refinar automáticamente las políticas de generación sin tocar pesos neuronales.

- **Fallo procedimental vs brecha de capacidad:** el LLM domina la sintaxis Java pero omite directivas de diseño; SkillOpt ataca la brecha de aplicación (*application gap*), elevando la fiabilidad en el dominio.
- **Qué es realmente una Habilidad (Skill):** no es un resumen informal, sino una política procedimental estricta con tres partes: **Trigger** (`when_to_apply`), **Procedimiento** (reglas ordenadas) y **Granularidad** (a nivel de tarea o evento).
- **Parámetro entrenable en el espacio de texto:** el documento de habilidad (300–2.000 tokens) se optimiza con rigor matemático sin reentrenar pesos neuronales ni incurrir en costes de GPU.
- **La condición crítica del oráculo:** «*un bucle sin diagnóstico detallado no supera a no tener bucle*» (CoEvoSkills: 41.1 con oráculo opaco vs 71.1 con verificador diagnóstico); por eso se ancla a trazas reales de `mvn test`.

Los 4 controles de fiabilidad en SkillOpt.

Gobernanza formal para evitar regresiones, alucinaciones y deriva descontrolada del manual.

CONTROL 1

Presupuesto Acotado ($\leq L_t = 4$)

Decaimiento coseno de ediciones permitidas por paso. Previene reescrituras caóticas y alucinaciones en el manual.

CONTROL 2

Held-out Validation Gate

Una propuesta de regla se acepta *si y solo si* mejora estrictamente la puntuación en tareas retenidas no vistas; empates son rechazados.

CONTROL 3

Búfer de Ediciones Rechazadas

Las propuestas fallidas se realimentan al optimizador como contraejemplos negativos, evitando ciclos y repetición de ideas descartadas.

CONTROL 4

Actualización Lenta / Meta-Habilidad

Visión longitudinal por épocas que consolida aprendizajes globales frente a la miopía de pasos locales individuales.

Evidencia empírica y resultados SOTA en SkillOpt.

Rendimiento estado del arte (SOTA) demostrado en benchmarks procedimentales rigurosos.

DOMINIO SOTA COMPROBADO

52 de 52 Celdas Líderes

- Mejor o empatado en el **100% de combinaciones** (modelo, benchmark, arnés).
- **GPT-5.5 direct chat: +23.5 pts** sobre base sin skill.
- **Codex: +24.8 pts · Claude Code: +19.1 pts.**
- Supera al oráculo por celda por **+5.4 pts**.
- **Economía de edición:** 1–4 ediciones aceptadas rinden **+29 a +39 pts**.
- **Transferencia cruzada:** Skill entrenada en Codex transfiere a Claude Code con **+59.7 pts** sobre baseline.

Componente Evaluado	Score	Impacto (Δ)
Full SkillOpt (SOTA)	77.5	—
Sin meta-habilidad	75.7	-1.8 pts
Sin meta + actualización lenta	55.0	-22.5 pts

Certeza empírica: La actualización longitudinal lenta es **estructuralmente indispensable**. Prescindir de ella degrada el sistema en 22.5 puntos.

El camino hacia la auto-mejora gobernada.

Hoja de ruta secuencial: de la estabilidad del verificador al bucle cerrado gobernado por SOTA.

Paso	Objetivo técnico	Esfuerzo / Coste
1	Verificador determinista: montaje hermético y resolución de reetiquetado SELinux :Z	Corto (1–2 días)
2	Canal de diagnóstico rico: minería estructurada de fallos en Surefire y trazas de compilación	Corto
3	Inyección y medición baseline: evaluar impacto de la habilidad Java/Maven inicial	\$3–\$5 en inferencia
4	Activación bucle SkillOpt: optimización reflexiva cerrada bajo los 4 controles de fiabilidad	Condicionado a pasos 1-3

De «no podía verificar su propia salida» a «**genera, compila y verifica por \$0.07, con ruta probada hacia la auto-mejora gobernada por SOTA**».